

CICD-ODI-Demo — Setup Guide

Terraform + GitHub Actions CI/CD around an ODI/MDLS Fivetran + dbt demo

Repo: github.com/fivetran-jasonchletsos/CICD-ODI-Demo · Built for the [#se_demo_improvements](#) effort

Do you need to edit the .tf files? No.

Every value that changes between SEs or accounts — bucket names, region, credentials, org name, whether the Snowflake consumer is on — is a Terraform variable. You only fill in `terraform.tfvars` and set matching GitHub secrets. Editing the `.tf` files themselves is only needed if you want to change the architecture, not to stand up this demo as it exists.

Zero-setup path

Forking or cloning the repo is enough to see it working: `dbt_run.yml` and `deploy.yml` already run green on push with no secrets configured, because DuckDB and the committed data snapshots are the always-on path. GitHub Pages already serves the frontend. Everything below is only for the live Fivetran/AWS path.

Live path — 7 steps

- 1 Fork or clone the repo**
Nothing to configure yet.
- 2 Copy `infra/terraform.tfvars.example` → `infra/terraform.tfvars`**
Fill in your Fivetran API key/secret, an external-ID string you pick, and (only if using it) Snowflake account details. AWS region and bucket names can stay default or change here — still no `.tf` file edits.
- 3 Add the same values as GitHub repo secrets**
Settings → Secrets and variables → Actions. See the table below for the full list.
- 4 Create a `production` GitHub Environment**
Settings → Environments, with at least one required reviewer. This is what makes `terraform-apply.yml` pause for a human click.
- 5 Run `bootstrap-mdls.yml` once**
Actions tab → Run workflow. Creates the Fivetran MDLS destination. Paste the printed id into `terraform.tfvars` and the `FIVETRAN_MDLS_DESTINATION_ID` secret.
- 6 Open a PR touching `infra/`, then merge it**
`terraform-plan.yml` posts a real plan on the PR. After merge, `terraform-apply.yml` pauses on the environment approval; once approved, it creates the S3 buckets, IAM roles, Glue databases, and the Fivetran connector. Add the resulting connector id as `FIVETRAN_CONNECTOR_ID`.

7 Demo it

Walk the GitHub Pages site (Architecture page for the diagram, Pipeline page for live workflow-run status, Dashboard for the marts), or open the AWS/Fivetran consoles to show what `terraform-apply.yml` created. `generate_and_sync.yml` keeps feeding new data through the real connector on a daily cron, or trigger it manually.

Required GitHub secrets

Secret	Used by	Notes
FIVETRAN_API_KEY	plan, apply, bootstrap, sync	Fivetran REST/Terraform API key
FIVETRAN_API_SECRET	plan, apply, bootstrap, sync	Fivetran REST/Terraform API secret
FIVETRAN_MDLS_DESTINATION_ID	plan, apply	From step 5 above
FIVETRAN_S3_EXTERNAL_ID	plan, apply	Same string as in terraform.tfvars
FIVETRAN_CONNECTOR_ID	generate_and_sync	From step 6, after first apply
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	generate_and_sync	Only needs s3:PutObject on the source bucket
SOURCE_BUCKET_NAME	generate_and_sync	Must match source_bucket_name in tfvars

Steps 3–7 need a real Fivetran account and an AWS account with permission to create S3 buckets, IAM roles, Glue databases, and an Athena workgroup. Skipping them is fine — the repo already demos the architecture and CI/CD pattern via the pushed frontend and workflow files with nothing configured. Full detail, including why each workflow exists, is in README.md.